

Homework 2

ASEN 5090 Introduction to GNSS

Justin Le

September 21, 2026

Problem 1

(a.)

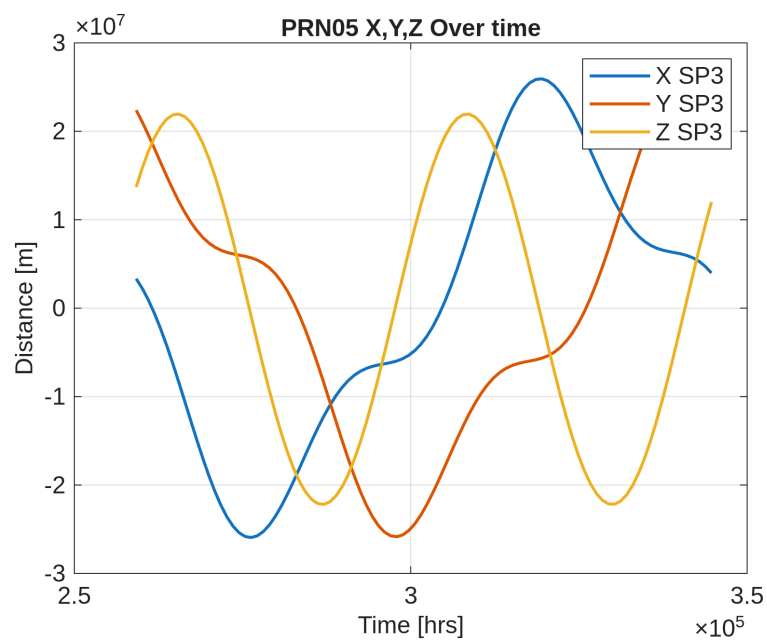


Figure 1: XYZ vs Time of SP3 Output

(b)

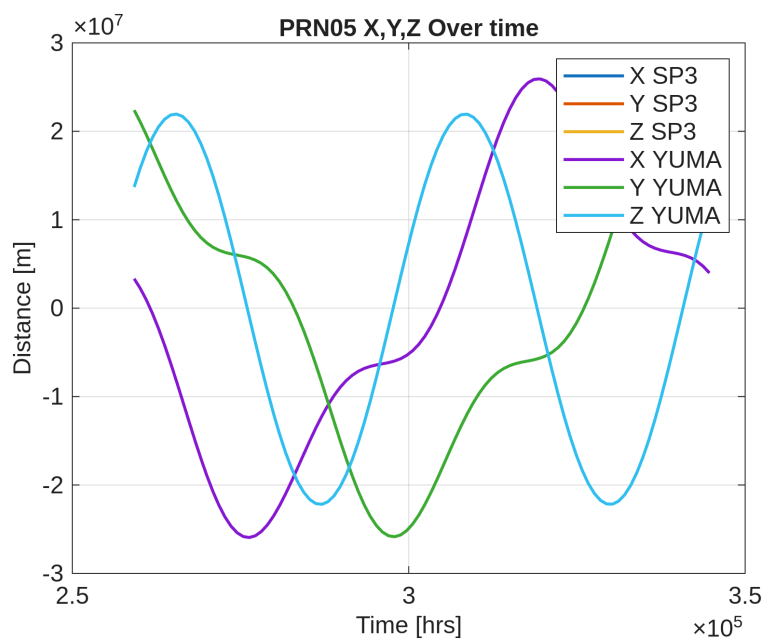


Figure 2: XYZ vs Time of SP3 Output with Almanac

(c.)

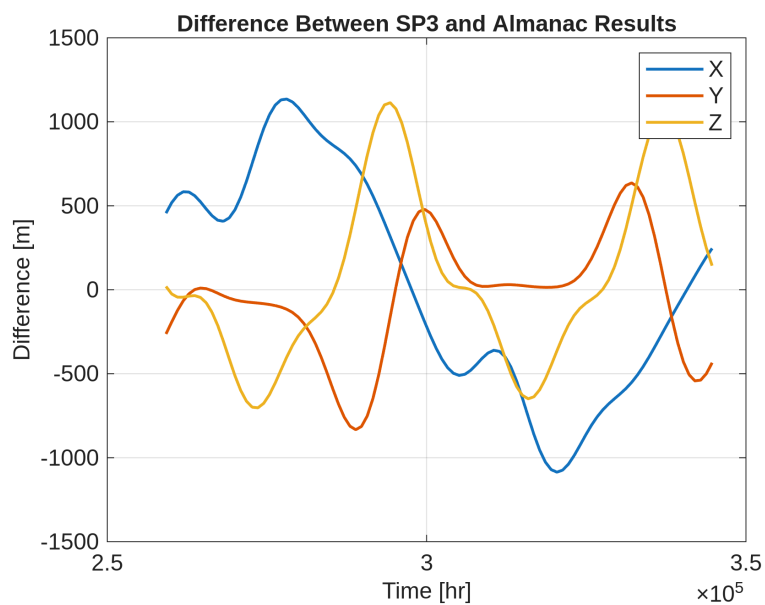


Figure 3: Residuals Between Almanac and SP3

(d.)

When observing the differences between the results of SP3 and the yuma almanac, if we treat SP3 data as our truth data we would have differences of up to 1 km as observed in Figure 3. Although large, if we consider the MEO orbit of the GPS constellations and the fact the observer will be on Earth, the differences in angles which we would use to determine if a satellite is visible would be less than a degree. This distance is not useful for tracking however very useful for checking if satellites are visible or not.

Problem 2

(a.)

Station	Lat (deg)	Lon (deg)	Height (m)	X (m)	Y (m)	Z (m)
NIST	39.995067	-105.262606	1648	-1288398	-4721697	4078625
SMEAD	40.010389	-105.244278	1588	-1286588	-4721009	4079890
EQUATOR	0	-105.262606	-18	-1678998	-6153159	0

Table 1: Site Locations

The LLA for NIST was given earlier. An arbitrary point on google maps for the Smead Aerospace backyard was selected, online tools were then used to approximate the altitude of this point chosen. The point at the equator was found by keeping the longitude coordinates of NIST but setting the latitude to 0. This is then examined on a map and then using online tools again approximate that coordinate's altitude.

(b)

Examine MATLAB function in MATLAB section below. We convert the ECEF vectors for the NIST site and site at the equator to unit vectors and then convert those unit vectors into ENU to get the following,

`r_enu_NIST =`

```

    0
-0.0033
1.0000
```

`r_enu_EQUATOR =`

```

    0
    0
1.0000
```

These are reasonable results for ENU, this is because if you imagine how the coordinate frames are set up, while in ECEF, any point on the surface would be entirely in the radial direction from

the center of the Earth. Therefore while on the surface, and vector that points to that site would be entirely in the up direction., The slight discrepancies that are seen in the NIST site are a result of discrepancies between the WGS84 model of the Earth and an actual sphere.

Problem 3

(a.)

The function is shown below, it is capable of using multiple rows of ECEF and calculating azimuth, elevation, and range. It is repeated again later in the MATLAB code section.

```
function [AZ, EL, RANGE] = compute_azelrange(userECEF, satECEF)
% satECEF is ECEF vector of sat
% userECEF is ECEF vector of the user

% Calculate LLA from userECEF
lla = ecef2lla(userECEF,'WGS84');
ref_lat = lla(1);
ref_lon = lla(2);
altitude = lla(3);
% Take difference of ECEF vectors, conver to ENU depending on ref angle
r_ECEF = satECEF-userECEF;

% Calculating range.
RANGE = sqrt(sum(r_ECEF.^2,2));

% Converting to ENU frame
r_ENU = (ECEF2ENU(ref_lat,ref_lon)*r_ECEF')';
x_E = r_ENU(:,1);
x_N = r_ENU(:,2);
x_U = r_ENU(:,3);

% Calculating azimuth and elevation
AZ = atan2d(x_E,x_N);
EL = asind(x_U./sqrt(sum(r_ENU.^2,2)));

end
```

We can actually check this function by emulating a satellite directly overhead NIST and feeding it into the function.

```
[az_test, el_test, range_test] = compute_azelrange(ecef_NIST, 5*ecef_NIST)

az_test =

    180.0000
```

```
el_test =  
  
    89.8107  
  
range_test =  
  
    2.5484e+07
```

From above we can see that the function works because with a satellite directly overhead, the function returns an elevation angle of 90 deg, and an azimuth of 180 deg. These angles geometrically make sense and the resulting range is 5 times the magnitude of NIST's ECEF vector which is also reasonable.

(b.)

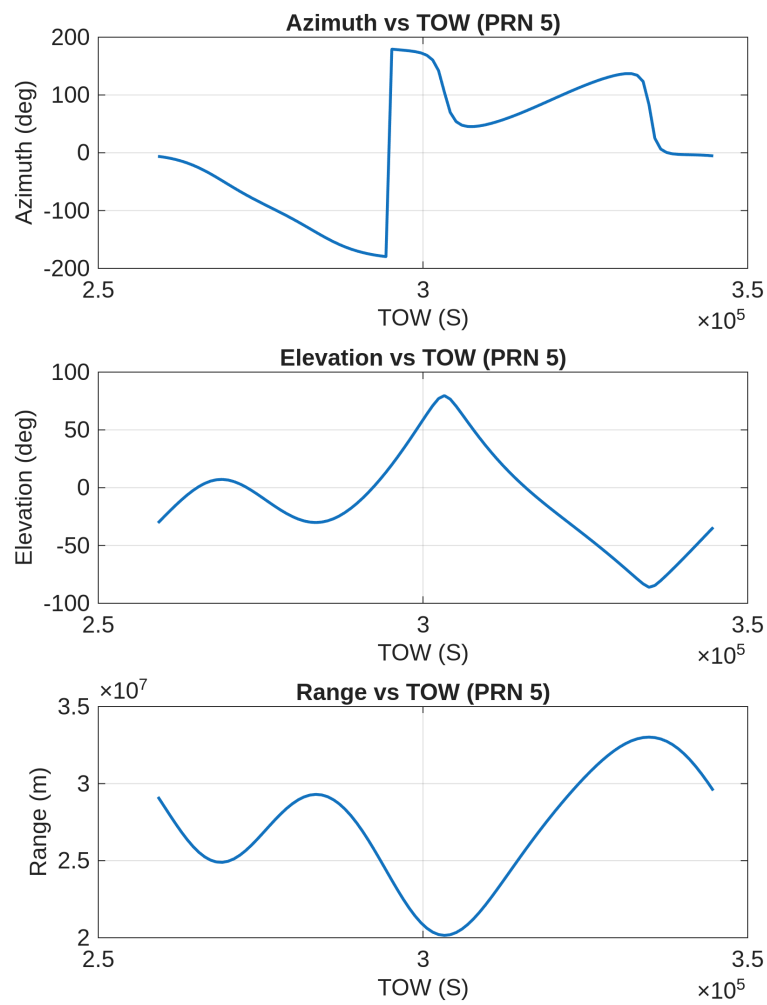


Figure 4: Azimuth, Elevation, Range for PRN5

(c.)

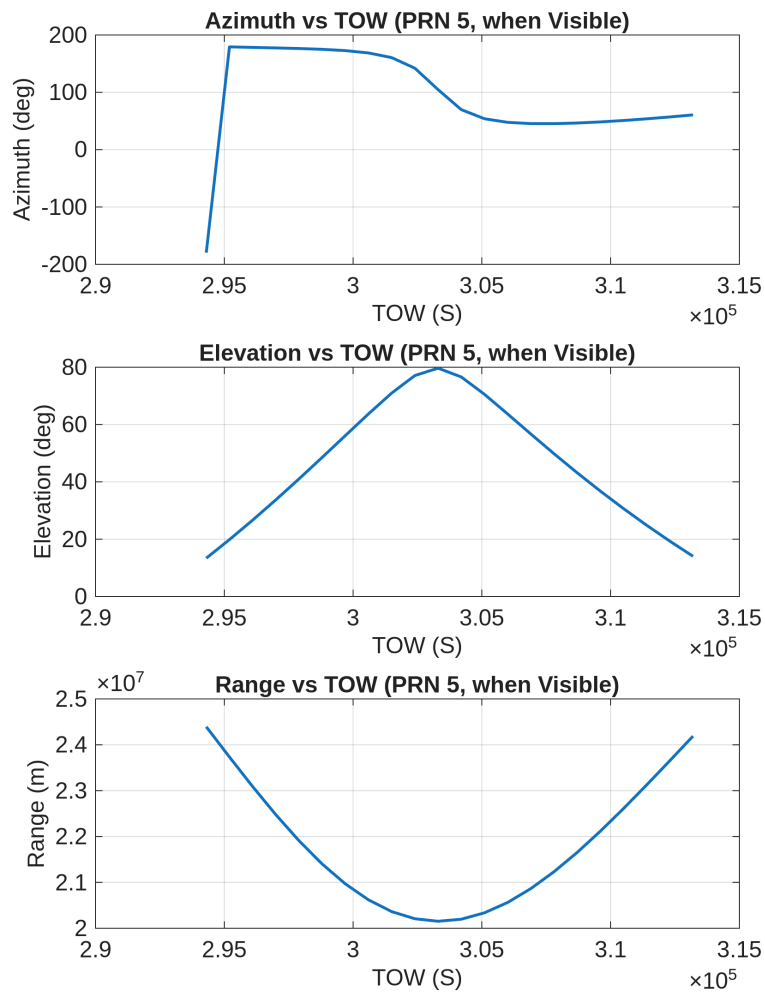


Figure 5: Azimuth, Elevation, Range for PRN5 when visible

Figure 5 is a result of applying an elevation mask of 10 deg. We are able to observe the results of only where the GPS satellite is 10 deg above the horizon. The graph shows this and that these occurrences are brief.

(d.)

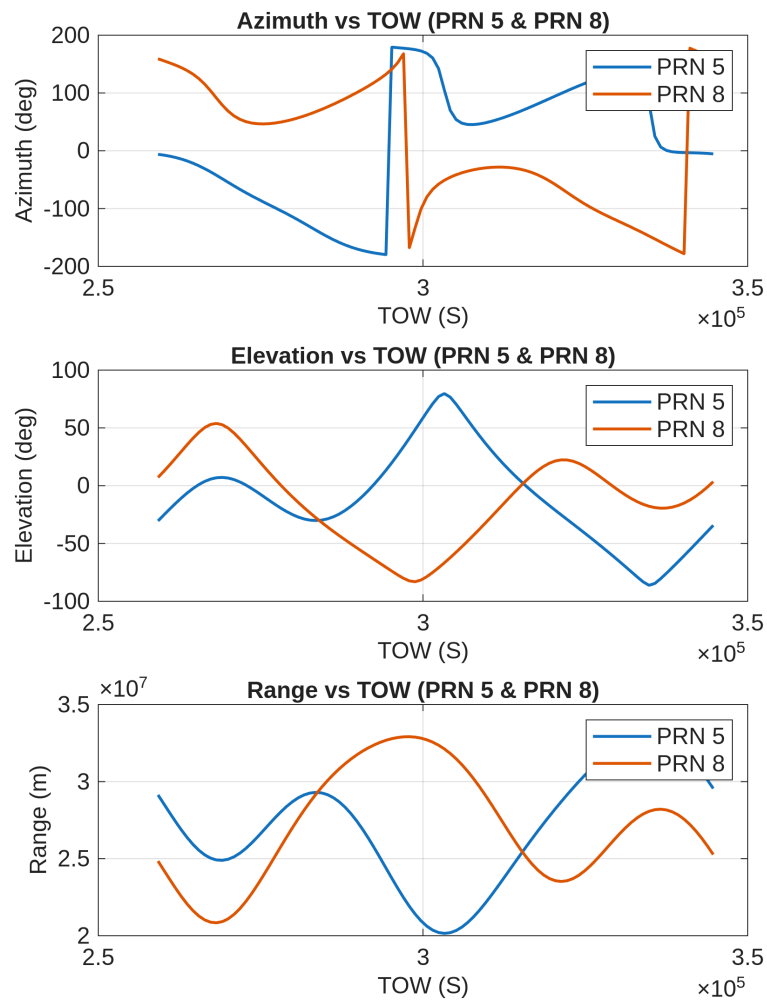


Figure 6: Azimuth, Elevation, Range for PRN5 and PRN8

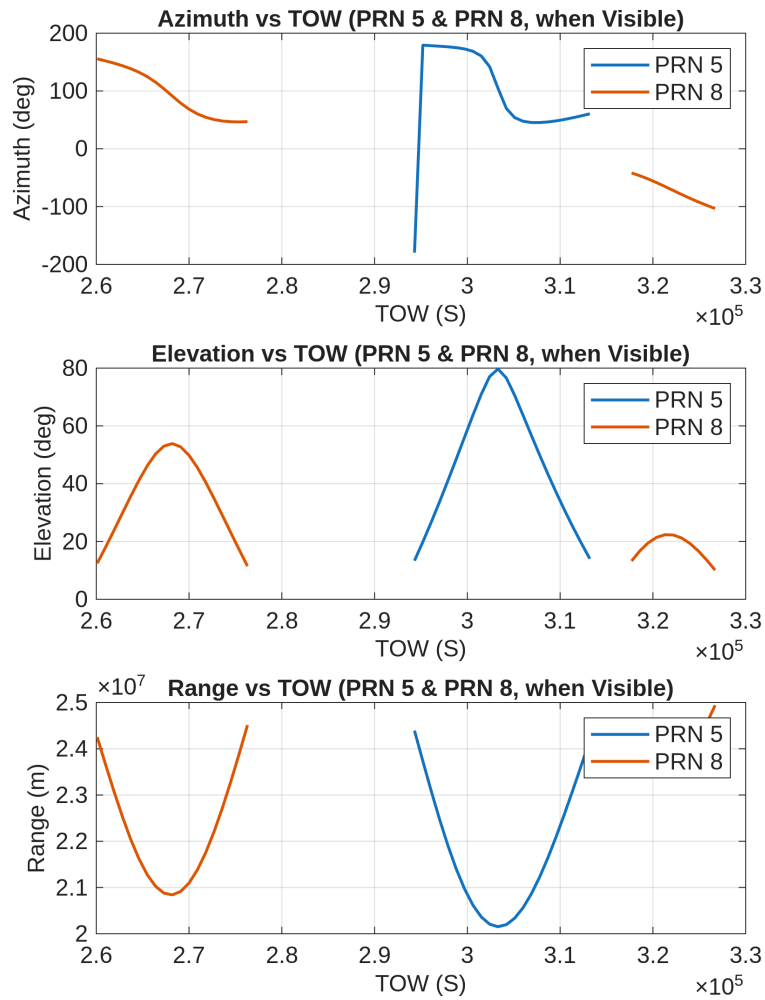


Figure 7: Azimuth, Elevation, Range for PRN5 and PRN8 when visible

From Figures 6 and 7, we can observe the differences in measurements for each GPS satellite and how they are visible at NIST. This is a result from the corresponding GPS satellites having different orbits, resulting in different measurements over time at any given site.

Problem 4

(a.)

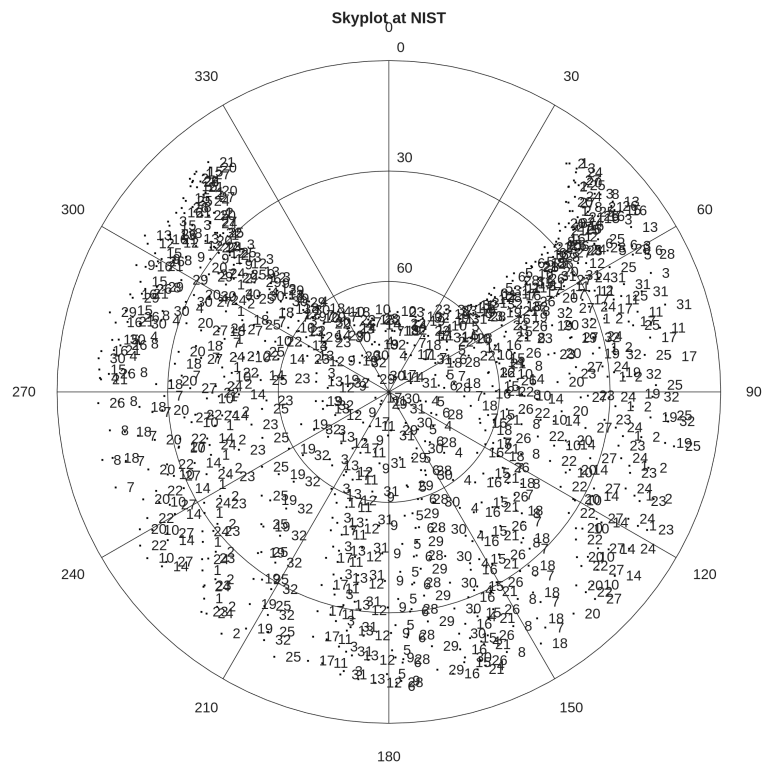


Figure 8: Skyplot at NIST

(b)

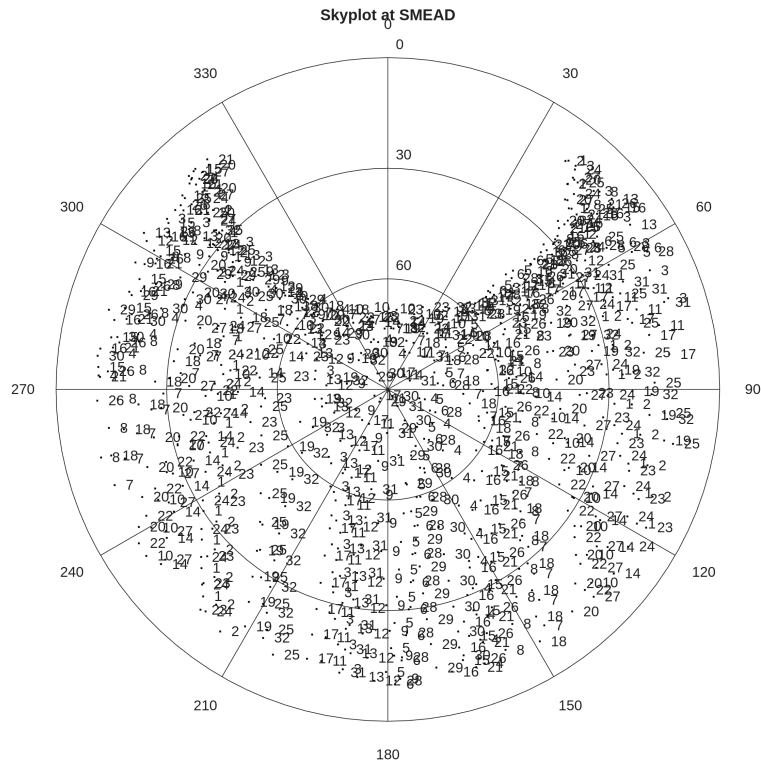


Figure 9: Skyplot at SMEAD

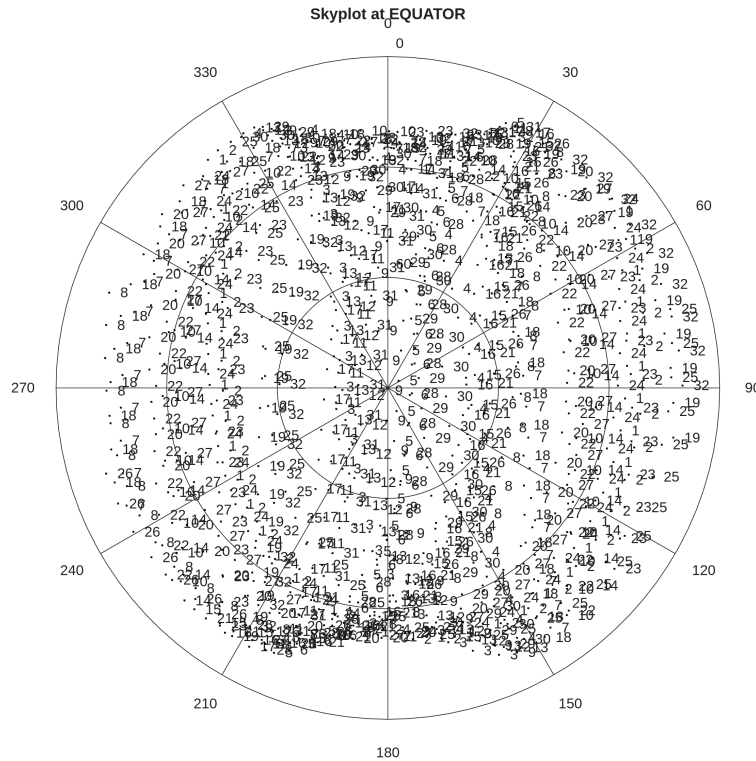


Figure 10: Skyplot at EQUATOR

(c.)

We are able to observe the differences in each Skyplot. NIST and SMEAD look strikingly similar with very small shifts in movements and which satellites are visible due to their close proximity to each other on Earth. This naturally results in skyplots that look similar, there would not be much change between the visibility when they are this close. In those skyplots we can observe a large gap in visibility above 60 deg in elevation between 320 deg and 40 deg in azimuth, likely a result from the satellite orbits and that specific location on Earth. There is greater difference when comparing the skyplot at the Equator. Here there is far more coverage, likely due to it being at the Equator and able to see more of the GPS constellation over time. This is not fully representative of what these sights will actually see because of this model neglecting environmental factors. For example, for the NIST and SMEAD sights, the literally big thing that this model does not account for in its elevation mask are the mountains. Mountains would obstruct some line of sight with the GPS satellites that would have been in view otherwise.

MATLAB Code

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW2 Part 1 Main Script

clc; clear; close all

% Reading in the sp3 file
sp3 = read_sp3('IGS00PSFIN_20262310000_01D_15M_ORB.SP3');

% Read entire spr for all prns and put into a struct
satPRN = extract_sp3_prn(sp3);

% Graphing PRN X,Y,Z [m] against time

% a.
figure()
plot(satPRN(5).time_epoch_hr,satPRN(5).X_m,'LineWidth',1.5)
hold on
plot(satPRN(5).time_epoch_hr,satPRN(5).Y_m,'LineWidth',1.5)
hold on
plot(satPRN(5).time_epoch_hr,satPRN(5).Z_m,'LineWidth',1.5)
title('PRN05 X,Y,Z Over time')
xlabel('Time [hrs]')
ylabel('Distance [m]')
grid on
hold on
legend('X SP3','Y SP3','Z SP3')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4.5]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'xyz_PRN5_sp3.png'), 'Resolution', 300);

% b. Using the almanac
[gps_ephem,gps_ephem_cell] = read_GPSyuma('YUMA231.alm.txt',2);

[health_PRN5,satPos_m_PRN5,satClkCorr_PRN5] = alm2pos(gps_ephem,[satPRN(5).week_number,
    satPRN(5).TOW_s] ,5);

plot(satPRN(5).TOW_s, satPos_m_PRN5, 'LineWidth',1.5)
legend('X SP3','Y SP3','Z SP3', 'X YUMA','Y YUMA','Z YUMA')
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4.5]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'xyz_PRN5_sp3_almanac.png'), 'Resolution', 300);

% c. Plot residuals
residuals_PRN5 = [satPos_m_PRN5(:,1)-satPRN(5).X_m,satPos_m_PRN5(:,2)-satPRN(5).Y_m,
    satPos_m_PRN5(:,3)-satPRN(5).Z_m];
figure()
plot(satPRN(5).time_epoch_hr,residuals_PRN5, 'LineWidth', 1.5)
title('Difference Between SP3 and Almanac Results')
xlabel('Time [hr]')
ylabel('Difference [m]')

```

```
legend('X','Y','Z')
grid on
set(gcf, 'Units', 'inches', 'Position', [0 0 6 4.5]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'prn5_residuals.png'), 'Resolution', 300);

% Export this for part 3.
save("HW2Part1.mat", 'satPRN')
```

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW2 Part 2 Main Script

clc; clear; close all
NIST = readtable('NIST_igs_station.csv');

% 3 User Positions in LLA

% NIST
lat_NIST = table2array(NIST(1,"Latitude"));
lon_NIST = table2array(NIST(1,"Longitude"));
height_m_NIST = table2array(NIST(1,"Height_m"));

lla_NIST = [lat_NIST, lon_NIST, height_m_NIST];

% SMEAD
lat_SMEAD = 40.010389;
lon_SMEAD = -105.244278;
height_m_SMEAD = 1603.553;
N_SMEAD = geoidheight(lat_SMEAD,lon_SMEAD); % Accounting for geoid
height_m_SMEAD = height_m_SMEAD + N_SMEAD;

lla_SMEAD = [lat_SMEAD, lon_SMEAD, height_m_SMEAD];

% EQUATOR
lat_EQUATOR = 0;
lon_EQUATOR = lon_NIST;
height_m_EQUATOR = geoidheight(lat_EQUATOR,lon_EQUATOR); % this is wrong?

lla_EQUATOR = [lat_EQUATOR, lon_EQUATOR, height_m_EQUATOR];

% 3 User Positions in ECEF
% Convert to ECEF
ecef_NIST = lla2ecef(lla_NIST, 'WGS84');
ecef_SMEAD = lla2ecef(lla_SMEAD, 'WGS84');
ecef_EQUATOR = lla2ecef(lla_EQUATOR, 'WGS84');

% Put into table for LLA and ECEF
Location = ["NIST"; "SMEAD"; "EQUATOR"];
Latitude_deg = [lat_NIST; lat_SMEAD; lat_EQUATOR];
Longitude_deg = [lon_NIST; lon_SMEAD; lon_EQUATOR];
Height_m = [height_m_NIST; height_m_SMEAD; height_m_EQUATOR];
X_m = [ecef_NIST(1); ecef_SMEAD(1); ecef_EQUATOR(1)];
Y_m = [ecef_NIST(2); ecef_SMEAD(2); ecef_EQUATOR(2)];
Z_m = [ecef_NIST(3); ecef_SMEAD(3); ecef_EQUATOR(3)];

%% Print table for reference
for i = 1:length(Location)
    fprintf('%-8s Lat: %.6f Lon: %.6f Height: %.0f X: %.0f Y: %.0f Z: %.0f\n', Location(i)
        ), Latitude_deg(i), Longitude_deg(i), Height_m(i), X_m(i), Y_m(i), Z_m(i));
end

```

% Part 2

% Finding reference position vector
% First get unit vectors of ECEF coords
r_ecef_NIST = ecef_NIST./norm(ecef_NIST);
r_ecef_EQUATOR = ecef_EQUATOR./norm(ecef_EQUATOR);
% Convert to ENU
r_enu_NIST = ECEF2ENU(lat_NIST,lon_NIST)*r_ecef_NIST'
r_enu_EQUATOR = ECEF2ENU(lat_EQUATOR,lon_EQUATOR)*r_ecef_EQUATOR'

% These make sense as the radial component of a position on the surface
% of the sphere should always relatively point up in ENU frame. Deviation
% is from variation between a sphere and wgs84 model.

% Export this for part 3.
save('user_ECEF.mat', 'ecef_NIST')
save('lat_NIST.mat', 'lat_NIST')
save('lon_NIST.mat', 'lon_NIST')

save('HW2Part2.mat', 'ecef_NIST', 'lat_NIST', 'lon_NIST', 'ecef_SMEAD', 'ecef_EQUATOR')

```

%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW2 Part 3 and 4 Main Script

clc; clear; close all

load("HW2Part1.mat")
load("HW2Part2.mat")

% Part a, write a function compute_azelrange, check with a simple source
% Force a satellite to be directly above NIST
[az_test, el_test, range_test] = compute_azelrange(ecef_NIST, 5*ecef_NIST)

% TODO: Explain that this is reasonable because elevation is ~90 degrees,
% directly over head which makes sense given the satellite being directly
% over head, range is also 5 times the magnitude of the NIST ecef vector which
% makes sense.

% Part b, plot ECEF for user at NIST for PRN5
[AZ_PRN5, EL_PRN5, RANGE_PRN5] = compute_azelrange(ecef_NIST, [satPRN(5).X_m, satPRN(5).
    Y_m, satPRN(5).Z_m]);

figure()
subplot(3,1,1)
plot(satPRN(5).TOW_s, AZ_PRN5, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Azimuth (deg)')
title('Azimuth vs TOW (PRN 5)')
grid on
subplot(3,1,2)
plot(satPRN(5).TOW_s, EL_PRN5, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Elevation (deg)')
title('Elevation vs TOW (PRN 5)')
grid on
subplot(3,1,3)
plot(satPRN(5).TOW_s, RANGE_PRN5, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Range (m)')
title('Range vs TOW (PRN 5)')
grid on
set(gcf, 'Units', 'inches', 'Position', [0 0 6 8]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'ranging_PRN5.png'), 'Resolution', 300);

% C. Only plot when NIST can view the satellite
% Filter when elevation > 10
visibleIndices_PRN5 = EL_PRN5>10;
TOW_s_PRN5_visible = satPRN(5).TOW_s;
AZ_PRN5_visible = AZ_PRN5; AZ_PRN5_visible(~visibleIndices_PRN5) = NaN;
EL_PRN5_visible = EL_PRN5; EL_PRN5_visible(~visibleIndices_PRN5) = NaN;
RANGE_PRN5_visible = RANGE_PRN5; RANGE_PRN5_visible(~visibleIndices_PRN5) = NaN;

AZ_PRN5(visibleIndices_PRN5)

```

```

figure()
subplot(3,1,1)
plot(TOW_s_PRN5_visible, AZ_PRN5_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Azimuth (deg)')
title('Azimuth vs TOW (PRN 5, when Visible)')
grid on
subplot(3,1,2)
plot(TOW_s_PRN5_visible, EL_PRN5_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Elevation (deg)')
title('Elevation vs TOW (PRN 5, when Visible)')
grid on
subplot(3,1,3)
plot(TOW_s_PRN5_visible, RANGE_PRN5_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Range (m)')
title('Range vs TOW (PRN 5, when Visible)')
grid on
set(gcf, 'Units', 'inches', 'Position', [0 0 6 8]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'ranging_PRN5_visibility.png'), 'Resolution',
    300);

% d. Repeat for PRN 8

[AZ_PRN8, EL_PRN8, RANGE_PRN8] = compute_azelrange(ecef_NIST, [satPRN(8).X_m, satPRN(8).
    Y_m, satPRN(8).Z_m]);

figure()
subplot(3,1,1)
plot(satPRN(5).TOW_s, AZ_PRN5, 'LineWidth', 1.5)
hold on
plot(satPRN(8).TOW_s, AZ_PRN8, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Azimuth (deg)')
title('Azimuth vs TOW (PRN 5 & PRN 8)')
legend('PRN 5', 'PRN 8')
grid on
subplot(3,1,2)
plot(satPRN(5).TOW_s, EL_PRN5, 'LineWidth', 1.5)
hold on
plot(satPRN(8).TOW_s, EL_PRN8, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Elevation (deg)')
title('Elevation vs TOW (PRN 5 & PRN 8)')
legend('PRN 5', 'PRN 8')
grid on
subplot(3,1,3)
plot(satPRN(5).TOW_s, RANGE_PRN5, 'LineWidth', 1.5)
hold on
plot(satPRN(8).TOW_s, RANGE_PRN8, 'LineWidth', 1.5)
xlabel('TOW (S)')

```

```

ylabel('Range (m)')
title('Range vs TOW (PRN 5 & PRN 8)')
legend('PRN 5', 'PRN 8')
grid on
set(gcf, 'Units', 'inches', 'Position', [0 0 6 8]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'ranging_PRN5_PRN8.png'), 'Resolution', 300);

% Only plot when NIST can view the satellite

visibleIndices_PRN8 = EL_PRN8>10;
TOW_s_PRN8_visible = satPRN(8).TOW_s;
AZ_PRN8_visible = AZ_PRN8; AZ_PRN8_visible(~visibleIndices_PRN8) = NaN;
EL_PRN8_visible = EL_PRN8; EL_PRN8_visible(~visibleIndices_PRN8) = NaN;
RANGE_PRN8_visible = RANGE_PRN8; RANGE_PRN8_visible(~visibleIndices_PRN8) = NaN;

figure()
subplot(3,1,1)
plot(TOW_s_PRN5_visible, AZ_PRN5_visible, 'LineWidth', 1.5)
hold on
plot(TOW_s_PRN8_visible, AZ_PRN8_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Azimuth (deg)')
title('Azimuth vs TOW (PRN 5 & PRN 8, when Visible)')
legend('PRN 5', 'PRN 8')
grid on
subplot(3,1,2)
plot(TOW_s_PRN5_visible, EL_PRN5_visible, 'LineWidth', 1.5)
hold on
plot(TOW_s_PRN8_visible, EL_PRN8_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Elevation (deg)')
title('Elevation vs TOW (PRN 5 & PRN 8, when Visible)')
legend('PRN 5', 'PRN 8')
grid on
subplot(3,1,3)
plot(TOW_s_PRN5_visible, RANGE_PRN5_visible, 'LineWidth', 1.5)
hold on
plot(TOW_s_PRN8_visible, RANGE_PRN8_visible, 'LineWidth', 1.5)
xlabel('TOW (S)')
ylabel('Range (m)')
title('Range vs TOW (PRN 5 & PRN 8, when Visible)')
legend('PRN 5', 'PRN 8')
grid on
set(gcf, 'Units', 'inches', 'Position', [0 0 6 8]);
set(findall(gcf, '-property', 'FontSize'), 'FontSize', 12);
exportgraphics(gcf, fullfile('figures', 'ranging_PRN5_PRN8_visible.png'), 'Resolution',
    300);

% Part 4: skyplots
% Calculate visibility for all PRNs at NIST
for prn = 1:32
    [satPRN(prn).AZ, satPRN(prn).EL, satPRN(prn).RANGE] = compute_azelrange(ecef_NIST, [
        satPRN(prn).X_m, satPRN(prn).Y_m, satPRN(prn).Z_m]);

```

```

    visibleIndices = satPRN(prn).EL > 10;
    satPRN(prn).AZ_visible = satPRN(prn).AZ; satPRN(prn).AZ_visible(~visibleIndices) =
        NaN;
    satPRN(prn).EL_visible = satPRN(prn).EL; satPRN(prn).EL_visible(~visibleIndices) =
        NaN;
end

figure()
for prn = 1:32
    plotAzEl(satPRN(prn).AZ_visible, satPRN(prn).EL_visible, satPRN(prn).PRN)
    hold on
end
title('Skyplot at NIST')
exportgraphics(gcf, fullfile('figures', 'NIST_skyplot.png'), 'Resolution', 300);
% Recalculate and plot for the two other sites

% SMEAD
for prn = 1:32
    [satPRN(prn).AZ, satPRN(prn).EL, satPRN(prn).RANGE] = compute_azelrange(ecef_SMEAD, [
        satPRN(prn).X_m, satPRN(prn).Y_m, satPRN(prn).Z_m]);
    visibleIndices = satPRN(prn).EL > 10;
    satPRN(prn).AZ_visible = satPRN(prn).AZ; satPRN(prn).AZ_visible(~visibleIndices) =
        NaN;
    satPRN(prn).EL_visible = satPRN(prn).EL; satPRN(prn).EL_visible(~visibleIndices) =
        NaN;
end

figure()
for prn = 1:32
    plotAzEl(satPRN(prn).AZ_visible, satPRN(prn).EL_visible, satPRN(prn).PRN)
    hold on
end
title('Skyplot at SMEAD')
exportgraphics(gcf, fullfile('figures', 'SMEAD_skyplot.png'), 'Resolution', 300);

% EQUATOR
for prn = 1:32
    [satPRN(prn).AZ, satPRN(prn).EL, satPRN(prn).RANGE] = compute_azelrange(ecef_EQUATOR,
        [satPRN(prn).X_m, satPRN(prn).Y_m, satPRN(prn).Z_m]);
    visibleIndices = satPRN(prn).EL > 10;
    satPRN(prn).AZ_visible = satPRN(prn).AZ; satPRN(prn).AZ_visible(~visibleIndices) =
        NaN;
    satPRN(prn).EL_visible = satPRN(prn).EL; satPRN(prn).EL_visible(~visibleIndices) =
        NaN;
end

figure()
for prn = 1:32
    plotAzEl(satPRN(prn).AZ_visible, satPRN(prn).EL_visible, satPRN(prn).PRN)
    hold on
end
title('Skyplot at EQUATOR')
exportgraphics(gcf, fullfile('figures', 'EQUATOR_skyplot.png'), 'Resolution', 300);

```

```
function C_ECEF2ENU = ECEF2ENU(ref_lat_deg, ref_lon_deg)

phi = ref_lat_deg;
lambda = ref_lon_deg;

R1 = [1 0 0;
      0 cosd(90-phi) sind(90-phi);
      0 -sind(90-phi) cosd(90-phi)];

R2 = [-sind(lambda) cosd(lambda) 0;
      -cosd(lambda) -sind(lambda) 0;
      0 0 1];

C_ECEF2ENU = R1*R2;

end
```

```
function [AZ, EL, RANGE] = compute_azelrange(userECEF, satECEF)
% satECEF is ECEF vector of sat
% userECEF is ECEF vector of the user

% Calculate LLA from userECEF
lla = ecef2lla(userECEF,'WGS84');
ref_lat = lla(1);
ref_lon = lla(2);
altitude = lla(3);
% Take difference of ECEF vectors, conver to ENU depending on ref angle
r_ECEF = satECEF-userECEF;

% Calculating range.
RANGE = sqrt(sum(r_ECEF.^2,2));

% Converting to ENU frame
r_ENU = (ECEF2ENU(ref_lat,ref_lon)*r_ECEF')';
x_E = r_ENU(:,1);
x_N = r_ENU(:,2);
x_U = r_ENU(:,3);

% Calculating azimuth and elevation
AZ = atan2d(x_E,x_N);
EL = asind(x_U./sqrt(sum(r_ENU.^2,2)));

end
```